

TIER 1 HELP DESK TASKS

Practice Guide & PowerShell Reference

corp.lab Active Directory Lab Environment

Ricardo Jaimes Hernandez

Azure Help Desk Lab Portfolio

How to Use This Guide

This guide is built for repetition. The goal is to develop muscle memory for the five most common Tier 1 Help Desk tasks so you can perform them without thinking during a real shift or an interview screen-share.

Practice Method

1. Read the scenario at the top of each task. Imagine the user is on the phone.
2. Do the GUI method first. It is slower but it builds the mental model of where things live in ADUC.
3. Then do the PowerShell method. It is faster but assumes you already know the model.
4. Read the interview talking points last. These are the answers when someone asks 'walk me through how you'd...'

Suggested Routine

- Week 1: Do all five tasks in GUI only. Do not worry about PowerShell yet.
- Week 2: Do all five tasks both ways. Time yourself.
- Week 3: PowerShell only. Try to do each task from memory without referring to the guide.
- Ongoing: Refresh once a month. Tier 1 muscle memory rusts fast.

Lab Setup Reminder

- Domain: corp.lab
- Domain Controller: DC01.corp.lab (10.10.1.4)
- Login: CORP\labadmin via RDP
- Tools open: Active Directory Users and Computers (ADUC) + PowerShell as Administrator

Test Users in the Lab

Username	Full Name	Department / OU	Security Group
mlopez	Maria Lopez	Helpdesk	HelpdeskAgents
jchen	James Chen	Helpdesk	HelpdeskAgents
spatel	Sarah Patel	Sales	SalesUsers
dnguyen	David Nguyen	Sales	SalesUsers
ajohnson	Alex Johnson	IT	ITAdmins
psingh	Priya Singh	IT	ITAdmins

Note: Default password for all test users is 'TempPass123!' (capital T, capital P, exclamation at the end). Some accounts have PasswordNeverExpires = \$true from the original setup, which conflicts with ChangePasswordAtLogon and is called out in Task 1.

Quick Reference Cheatsheet

When you only have 30 seconds before someone asks how to do something, this is the page.

Task	GUI (ADUC)	PowerShell
Password reset	Right-click user > Reset Password	<code>Set-ADAccountPassword + Set-ADUser - ChangePasswordAtLogon \$true</code>
Account unlock	Right-click user > Properties > Account > Unlock	<code>Unlock-ADAccount</code>
Add to group	Right-click user > Add to a group	<code>Add-ADGroupMember</code>
Disable account	Right-click user > Disable Account	<code>Disable-ADAccount</code>
Move between OUs	Right-click user > Move...	<code>Move-ADObject</code>

Task 1: Password Reset

Scenario

A user calls saying they forgot their password and cannot log in. You need to reset it to a temporary value and force them to change it at next login. This is the #1 Help Desk request, often 30-40% of all Tier 1 tickets.

Before you reset: Always verify the user's identity first. Knowledge questions, callback to their listed number, or in-person verification depending on company policy. Never reset based on an emailed request alone.

GUI Method (ADUC)

1. In ADUC, expand corp.lab > Sales OU.
2. Right-click Sarah Patel > Reset Password...
3. Enter a new temporary password (e.g., TempReset2026!) in both fields.
4. Check the box: User must change password at next login.
5. Click OK.
6. Confirmation popup appears: 'The password for Sarah Patel has been changed.'

PowerShell Method

```
# Reset Sarah's password and require change at next login
$newPass = ConvertTo-SecureString "TempReset2026!" -AsPlainText -Force
Set-ADAccountPassword -Identity "spatel" -NewPassword $newPass -Reset

# Clear PasswordNeverExpires first (lab accounts have this set)
Set-ADUser -Identity "spatel" -PasswordNeverExpires $false
Set-ADUser -Identity "spatel" -ChangePasswordAtLogon $true

# Verify
Get-ADUser -Identity "spatel" -Properties PasswordLastSet, PasswordExpired,
PasswordNeverExpires |
    Select-Object SamAccountName, PasswordLastSet, PasswordExpired,
PasswordNeverExpires
```

Expected Output

SamAccountName	PasswordLastSet	PasswordExpired	PasswordNeverExpires
spatel	6/7/2026 12:02:46 AM	True	False

Interview Talking Points

- I always verify the user's identity before resetting. Knowledge questions, callback to their listed number, or in-person.

- I prefer GUI for one-off resets where I want to see the user's other attributes. PowerShell for bulk operations or scripting.
- Forcing change at next login is standard. It means even I as the admin do not know their final password.
- Gotcha I hit in my own lab: PasswordNeverExpires conflicts with ChangePasswordAtLogon. You have to clear the never-expires flag first.

Task 2: Account Unlock

Scenario

A user calls saying they cannot log in, getting locked out. They typed their password wrong too many times. You need to unlock the account without changing the password.

Lab pre-requisite: Out of the box, AD ships with LockoutThreshold = 0, meaning lockouts are disabled. Configure a real policy first (this is itself a hardening task).

Set the Lockout Policy (one-time setup)

```
# Configure: lock after 5 failed attempts, 15-minute lockout
Set-ADDefaultDomainPasswordPolicy -Identity "corp.lab" `
  -LockoutThreshold 5 `
  -LockoutDuration 00:15:00 `
  -LockoutObservationWindow 00:15:00

# Verify
Get-ADDefaultDomainPasswordPolicy |
  Select-Object LockoutThreshold, LockoutObservationWindow, LockoutDuration
```

Simulate a Lockout (for practice)

```
$wrongPass = ConvertTo-SecureString "BadPassword" -AsPlainText -Force
$creds = New-Object System.Management.Automation.PSCredential("CORP\jchen",
$wrongPass)
1..6 | ForEach-Object {
    Start-Process -FilePath "whoami" -Credential $creds -ErrorAction
SilentlyContinue
    Start-Sleep -Milliseconds 300
}
Start-Sleep -Seconds 2

# Confirm jchen is locked
Get-ADUser -Identity "jchen" -Properties LockedOut, AccountLockoutTime |
  Select-Object SamAccountName, LockedOut, AccountLockoutTime
```

GUI Method (ADUC)

1. In ADUC, navigate to Helpdesk OU.
2. Right-click James Chen > Properties.
3. Click the Account tab.
4. Look for the checkbox: *'Unlock account. This account is currently locked out on this Active Directory Domain Controller.'*
5. Check the box.

6. Click OK.

PowerShell Method

```
# Unlock the account
Unlock-ADAccount -Identity "jchen"

# Verify
Get-ADUser -Identity "jchen" -Properties LockedOut |
    Select-Object SamAccountName, LockedOut
```

Interview Talking Points

- Lockouts are usually self-inflicted. User mistypes their password or has an old cached credential on their phone or tablet.
- Repeated lockouts on the same account warrant investigation. Could indicate credential stuffing, brute force, or a service account running with an old password.
- Before unlocking, I check the Security event log to see where the failed attempts came from. If from an unexpected machine or IP, escalate.
- After unlock, ask the user where they were trying to log in from. Phone email apps are the #1 culprit for 'mystery' lockouts after a password change.
- In my lab specifically: I found AD ships with lockout disabled. I applied a 5-attempt / 15-minute policy first as a baseline hardening step.

Task 3: Add User to a Security Group

Scenario

David Nguyen (Sales) needs access to the Sales Reports shared folder. Permissions are granted through the SalesUsers group, so you need to add him.

Real-world note: Group membership requests usually require manager approval. Verify the approval ticket before making the change.

Setup (remove David first so you can add him back)

```
# Check current membership
Get-ADGroupMember -Identity "SalesUsers" | Select-Object Name, SamAccountName

# Remove David for practice
Remove-ADGroupMember -Identity "SalesUsers" -Members "dnguyen" -Confirm:$false

# Confirm he is out
Get-ADGroupMember -Identity "SalesUsers" | Select-Object Name
```

GUI Method (ADUC)

1. In ADUC, navigate to Sales OU.
2. Right-click David Nguyen > Add to a group...
3. Type SalesUsers in the dialog box.
4. Click Check Names. The name should underline confirming it was found.
5. Click OK. Confirmation popup appears.
6. To verify: right-click David Nguyen > Properties > Member Of tab. Should show Domain Users and SalesUsers.

PowerShell Method

```
# Add David to SalesUsers
Add-ADGroupMember -Identity "SalesUsers" -Members "dnguyen"

# Verify
Get-ADGroupMember -Identity "SalesUsers" | Select-Object Name

# Alternative: check user's group memberships
Get-ADPrincipalGroupMembership -Identity "dnguyen" | Select-Object Name
```

Interview Talking Points

- Group membership is how access is granted. You almost never assign permissions directly to a user account in a well-run environment.

- If a user 'needs access to a folder,' the real question is 'which group has that access?' Find the group, add the user.
- In real environments, this kind of request needs manager approval. Help Desk verifies the approval ticket before making the change.
- Add-ADGroupMember and Add-LocalGroupMember are different. The first is for AD groups, the second is for local groups on a specific machine.

Task 4: Disable a User Account

Scenario

Alex Johnson (IT) is going on a 6-month sabbatical. HR has requested the account be disabled to prevent access during leave, but not deleted (his data and historical activity need to stay accessible).

Disable before delete: Disable preserves the SID and all historical references (NTFS permissions, event logs). Deletion makes audit trails harder to interpret. Standard policy: disable on last day, delete 30-90 days later after data archival.

GUI Method (ADUC)

1. In ADUC, navigate to IT OU.
2. Right-click Alex Johnson > Disable Account.
3. Confirmation popup appears > Yes.
4. The user's icon now shows a downward arrow indicating disabled.

To Re-Enable

5. Right-click Alex Johnson > Enable Account.

PowerShell Method

```
# Disable the account
Disable-ADAccount -Identity "ajohnson"

# Verify
Get-ADUser -Identity "ajohnson" -Properties Enabled |
    Select-Object SamAccountName, Enabled

# Re-enable when needed
Enable-ADAccount -Identity "ajohnson"

# Confirm re-enabled
Get-ADUser -Identity "ajohnson" -Properties Enabled |
    Select-Object SamAccountName, Enabled
```

Interview Talking Points

- Disable before delete is best practice. Deleting an account destroys the SID, and historical NTFS permissions or event logs referencing that user become harder to interpret.
- Common triggers: employee on extended leave, terminated employee, contractor end-of-engagement.
- Standard policy: disable on the employee's last day. Delete 30-90 days later after data archival.
- If a terminated employee's data needs to be retained, sometimes you also reset their password and forward their email before disabling.

Task 5: Move User Between OUs

Scenario

Maria Lopez transferred from Helpdesk to IT. You need to move her account from the Helpdesk OU to the IT OU. This matters because OU membership controls Group Policy application.

Why OUs matter: OUs are how Group Policy gets applied. If Sales has policies that lock down USB ports and you move someone there, they inherit those restrictions immediately on next login. Department transfers often require updating group memberships too, not just the OU.

GUI Method (ADUC)

1. In ADUC, navigate to Helpdesk OU.
2. Right-click Maria Lopez > Move...
3. In the popup, navigate to corp.lab > IT > OK.
4. Maria now appears in the IT OU, no longer in Helpdesk.

Move Her Back

5. In IT OU, right-click Maria Lopez > Move... > back to Helpdesk.

PowerShell Method

```
# Get Maria's current Distinguished Name
$user = Get-ADUser -Identity "mlopez"
Write-Host "Current location: $($user.DistinguishedName)" -ForegroundColor Yellow

# Move to IT OU
Move-ADObject -Identity $user.DistinguishedName -TargetPath
"OU=IT,DC=corp,DC=lab"

# Verify new location
$user = Get-ADUser -Identity "mlopez"
Write-Host "New location: $($user.DistinguishedName)" -ForegroundColor Green

# Move back
Move-ADObject -Identity $user.DistinguishedName -TargetPath
"OU=Helpdesk,DC=corp,DC=lab"
Write-Host "Restored to Helpdesk OU" -ForegroundColor Cyan
```

Interview Talking Points

- OU placement is more important than people realize. OUs are how Group Policy gets applied.
- Department transfers involve more than moving the user. Old group memberships should be removed, new ones added.
- Some shops automate OU moves via PowerShell scripts triggered by HR ticketing systems.

- Move-ADObject works on any AD object, not just users.

Appendix A: PowerShell Cheatsheet

Reference for the AD PowerShell module. Memorize the verbs (Get, Set, New, Remove, Add, Move, Enable, Disable, Unlock) and the nouns become predictable.

User Management

```
# Find a user
Get-ADUser -Identity "username"
Get-ADUser -Filter "Name -like '*lopez*'"

# Properties (default returns limited fields - use -Properties for more)
Get-ADUser -Identity "spatel" -Properties *
Get-ADUser -Identity "spatel" -Properties LastLogon, LockedOut, PasswordLastSet

# Reset password
$pw = ConvertTo-SecureString "NewPass2026!" -AsPlainText -Force
Set-ADAccountPassword -Identity "spatel" -NewPassword $pw -Reset

# Force password change at next logon
Set-ADUser -Identity "spatel" -PasswordNeverExpires $false
Set-ADUser -Identity "spatel" -ChangePasswordAtLogon $true

# Enable / Disable
Enable-ADAccount -Identity "spatel"
Disable-ADAccount -Identity "spatel"

# Unlock
Unlock-ADAccount -Identity "spatel"

# Create new user (basic)
New-ADUser -Name "John Doe" -SamAccountName "jdoe" `
  -UserPrincipalName "jdoe@corp.lab" `
  -Path "OU=Sales,DC=corp,DC=lab" `
  -AccountPassword $pw -Enabled $true

# Delete user (use with caution)
Remove-ADUser -Identity "jdoe" -Confirm:$true
```

Group Management

```
# List group members
Get-ADGroupMember -Identity "SalesUsers"

# What groups is a user in?
Get-ADPrincipalGroupMembership -Identity "spatel" | Select Name

# Add / Remove
Add-ADGroupMember -Identity "SalesUsers" -Members "dnguyen"
```

```
Remove-ADGroupMember -Identity "SalesUsers" -Members "dnguyen" -Confirm:$false

# Create new group
New-ADGroup -Name "MarketingUsers" `
  -GroupScope Global -GroupCategory Security `
  -Path "OU=Sales,DC=corp,DC=lab"
```

OU Management

```
# List OUs
Get-ADOrganizationalUnit -Filter * | Select Name, DistinguishedName

# Move object between OUs
$user = Get-ADUser -Identity "mlopez"
Move-ADObject -Identity $user.DistinguishedName `
  -TargetPath "OU=IT,DC=corp,DC=lab"

# Create new OU
New-ADOrganizationalUnit -Name "Marketing" `
  -Path "DC=corp,DC=lab" `
  -ProtectedFromAccidentalDeletion $false
```

Policy & Diagnostics

```
# Check domain password policy
Get-ADDefaultDomainPasswordPolicy

# Set lockout policy
Set-ADDefaultDomainPasswordPolicy -Identity "corp.lab" `
  -LockoutThreshold 5 `
  -LockoutDuration 00:15:00 `
  -LockoutObservationWindow 00:15:00

# Find all locked-out accounts
Search-ADAccount -LockedOut

# Find disabled accounts
Search-ADAccount -AccountDisabled

# Find accounts inactive for 90+ days
Search-ADAccount -AccountInactive -TimeSpan 90.00:00:00
```

Useful Get-WinEvent Patterns

```
# Last 10 failed logons (Event 4625)
Get-WinEvent -LogName Security -MaxEvents 10 `
  -FilterXPath "[*][System[EventID=4625]]" |
```

```
Select TimeCreated, Id, Message
```

```
# Last password resets (Event 4724)
```

```
Get-WinEvent -LogName Security -MaxEvents 10 `
  -FilterXPath "[System[EventID=4724]]"
```

```
# Group membership changes (4728/4732)
```

```
Get-WinEvent -LogName Security -MaxEvents 10 `
  -FilterXPath "[System[EventID=4728 or EventID=4732]]"
```

```
# Enable auditing categories
```

```
auditpol /set /subcategory:"Logon" /success:enable /failure:enable
```

```
auditpol /set /subcategory:"User Account Management" /success:enable
/failure:enable
```

```
auditpol /get /category:"Logon/Logoff"
```

PowerShell Fundamentals (refresher)

```
# Variables
```

```
$user = Get-ADUser -Identity "spatel"
```

```
$user.Name           # property access
```

```
$user | Get-Member    # explore object
```

```
# Pipeline
```

```
Get-ADUser -Filter * | Where-Object { $_.Enabled -eq $true } | Select Name
```

```
# Common shorthand
```

```
$_ = the current pipeline object
```

```
ft = Format-Table
```

```
fl = Format-List
```

```
? = Where-Object
```

```
% = ForEach-Object
```

```
# Help (always available)
```

```
Get-Help Set-ADUser -Examples
```

```
Get-Command -Module ActiveDirectory
```

```
Get-Help Set-ADUser -Online
```

Appendix B: KQL Queries for SOC Monitoring

Run these in Log Analytics workspace law-helpdesk-lab. Events from DC01 land in the Event table (legacy schema from Azure Monitor Agent DCR).

Query 1: Failed Logon Attempts

```
// Purpose: Identify failed authentication attempts. Failed logons can
//           indicate brute force, password spraying, stale credentials,
//           or simply users mistyping their password.
// Tier 1 action: Verify the failures match the user's expected activity.
//           If unfamiliar, escalate to Tier 2 / Security.

Event
| where EventID == 4625
| project TimeGenerated, Computer, EventID, RenderedDescription
| order by TimeGenerated desc
| take 10
```

Query 2: Password Resets Audit Trail

```
// Purpose: Audit admin-initiated password resets (4724).
//           Self-service password changes are 4723, different event.
// Tier 1 action: Confirm each reset matches an open ticket.

Event
| where EventID == 4724
| project TimeGenerated, Computer, RenderedDescription
| order by TimeGenerated desc
| take 10
```

Query 3: Brute Force Detection

```
// Purpose: Detect potential brute force by counting failed attempts
//           per computer over a short time window.
// Detection logic:
// 1. Filter to failed logons (4625)
// 2. Group by source computer
// 3. Flag any computer with 3+ failures
// 4. Calculate time window - short windows = higher confidence

Event
| where EventID == 4625
| summarize FailedAttempts = count(),
             FirstAttempt = min(TimeGenerated),
             LastAttempt = max(TimeGenerated)
             by Computer
```



```
| where FailedAttempts >= 3
| extend TimeWindowSeconds = datetime_diff('second', LastAttempt, FirstAttempt)
| project Computer, FailedAttempts, FirstAttempt, LastAttempt, TimeWindowSeconds
| order by FailedAttempts desc
```

Query 4: Group Membership Changes

```
// Purpose: Track members added to security groups.
//          Privilege escalation often shows up here first.
// EventIDs: 4728 = global group, 4732 = local group, 4756 = universal
// Tier 1 action: Cross-reference against approved change tickets.
```

```
Event
| where EventID in (4728, 4732, 4756)
| project TimeGenerated, Computer, EventID, RenderedDescription
| order by TimeGenerated desc
| take 20
```

Query 5: Event Type Profile

```
// Purpose: Profile what events are being collected.
//          Useful for verifying audit policy and spotting unusual spikes.
```

```
Event
| summarize EventCount = count(), MostRecent = max(TimeGenerated) by EventID
| extend EventDescription = case(
    EventID == 4624, "Successful logon",
    EventID == 4625, "Failed logon",
    EventID == 4634, "Logoff",
    EventID == 4672, "Special privileges assigned",
    EventID == 4673, "Sensitive privilege use",
    EventID == 4688, "Process creation",
    EventID == 4724, "Password reset (admin-initiated)",
    EventID == 4728, "Member added to global group",
    EventID == 4732, "Member added to local group",
    EventID == 4740, "Account locked out",
    "Other"
)
| project EventID, EventDescription, EventCount, MostRecent
| order by EventCount desc
```

Appendix C: Key Security Event IDs

Memorize these. They come up in interviews constantly, and they are how you read a Windows security log.

Event ID	Name	What it means / why it matters
4624	Successful logon	Normal authentication. Spikes on a single account may indicate compromise.
4625	Failed logon	Most common SOC alert source. Multiple = brute force / password spray.
4634	Logoff	Pair with 4624 for session duration analysis.
4672	Special privileges assigned	Admin or privileged account logged on. Watch for unexpected accounts.
4673	Sensitive privilege use	Often noisy. Useful for detecting privilege abuse.
4688	Process creation	Tracks what programs run. Key for malware investigation.
4720	User account created	Should match an HR onboarding ticket.
4724	Password reset (admin)	Help Desk activity. Should match a ticket. Watch resets on privileged accounts.
4726	User account deleted	Should match HR offboarding. Investigate unexpected deletions.
4728	Added to global security group	Privilege escalation indicator if the group is privileged.
4732	Added to local security group	Local admin add - watch for additions to local Administrators.
4738	User account changed	Any attribute change. Detail in Message body.
4740	Account locked out	Often a self-inflicted user mistake, but repeated = attack indicator.
4756	Added to universal security group	Similar to 4728 but for universal groups across multi-domain forests.
4767	Account unlocked	Help Desk activity. Cross-reference with 4740 to confirm valid resolution.

Appendix D: corp.lab Seed Script

This is the PowerShell script that rebuilds the OU/group/user structure if you ever need to wipe and restart. Kept as an artifact even though we used ADUC for the original build, because automating user provisioning is itself a relevant skill.

How to use: Save this as C:\Scripts\seed-corp-lab.ps1 on DC01. Run from an elevated PowerShell session after AD DS is installed and the domain is promoted.

```
<#
.SYNOPSIS
    Seeds the corp.lab Active Directory environment with OUs,
    security groups, and test users.

.DESCRIPTION
    Creates three OUs (Helpdesk, Sales, IT), one security group per OU,
    and six test users. Idempotent: safe to re-run.

.NOTES
    Author: Ricardo Jaimes Hernandez
    Project: Azure Help Desk Portfolio Lab
#>

$domain = "corp.lab"
$dn = "DC=corp,DC=lab"
$defaultPassword = ConvertTo-SecureString "TempPass123!" -AsPlainText -Force

# Create OUs
$ous = @("Helpdesk", "Sales", "IT")
foreach ($ou in $ous) {
    if (-not (Get-ADOrganizationalUnit -Filter "Name -eq '$ou'" -ErrorAction
SilentlyContinue)) {
        New-ADOrganizationalUnit -Name $ou -Path $dn -
ProtectedFromAccidentalDeletion $false
        Write-Host "Created OU: $ou" -ForegroundColor Green
    }
}

# Create security groups (one per OU)
$groups = @{
    "HelpdeskAgents" = "Helpdesk"
    "SalesUsers"      = "Sales"
    "ITAdmins"        = "IT"
}
foreach ($groupName in $groups.Keys) {
    $ouPath = "OU=($($groups[$groupName]),$dn"
    if (-not (Get-ADGroup -Filter "Name -eq '$groupName'" -ErrorAction
SilentlyContinue)) {
        New-ADGroup -Name $groupName -GroupScope Global -GroupCategory Security
-Path $ouPath
        Write-Host "Created Group: $groupName" -ForegroundColor Green
    }
}
```

```

    }
}

# Create users
$users = @(
    @{First="Maria"; Last="Lopez"; Dept="Helpdesk"; Group="HelpdeskAgents";
    Title="Help Desk Technician"},
    @{First="James"; Last="Chen"; Dept="Helpdesk"; Group="HelpdeskAgents";
    Title="Help Desk Technician"},
    @{First="Sarah"; Last="Patel"; Dept="Sales"; Group="SalesUsers";
    Title="Account Executive"},
    @{First="David"; Last="Nguyen"; Dept="Sales"; Group="SalesUsers";
    Title="Sales Representative"},
    @{First="Alex"; Last="Johnson"; Dept="IT"; Group="ITAdmins";
    Title="Systems Administrator"},
    @{First="Priya"; Last="Singh"; Dept="IT"; Group="ITAdmins";
    Title="IT Manager"}
)

foreach ($u in $users) {
    $sam = ($u.First.Substring(0,1) + $u.Last).ToLower()
    $upn = "$sam@$domain"
    $ouPath = "OU= $($u.Dept), $dn"

    if (-not (Get-ADUser -Filter "SamAccountName -eq '$sam'" -ErrorAction
    SilentlyContinue)) {
        New-ADUser `
            -Name "$($u.First) $($u.Last)" `
            -GivenName $u.First `
            -Surname $u.Last `
            -SamAccountName $sam `
            -UserPrincipalName $upn `
            -DisplayName "$($u.First) $($u.Last)" `
            -Title $u.Title `
            -Department $u.Dept `
            -Path $ouPath `
            -AccountPassword $defaultPassword `
            -Enabled $true `
            -ChangePasswordAtLogon $false

        Add-ADGroupMember -Identity $u.Group -Members $sam
        Write-Host "Created user: $sam" -ForegroundColor Green
    }
}

Write-Host "`n=== Seed complete ===" -ForegroundColor Cyan
Write-Host "Default password: TempPass123!" -ForegroundColor Cyan

```